

# PRÁCTICA T.5

**Jaume Adrover Fernández**  
**43235882E**

## **Bloque obligatorio**

Esta práctica consiste en implementar y verificar el correcto funcionamiento del algoritmo del valor medio para resolver redes cerradas (MVA). El algoritmo se puede encontrar en la página 136 del libro de la asignatura. También, se pueden hacer uso de los ejercicios resueltos en el libro para comprobar que el algoritmo funciona correctamente.

Además, se deberán mostrar gráficamente el valor de las variables  $R_i$ ,  $R$ ,  $X$ ,  $N_i$ ,  $X_i$  y  $U_i$  en función del número de clientes que hay en el sistema.

Se deberá entregar:

- Implementación del algoritmo en el lenguaje que el alumno considere más oportuno.
- Un documento donde se reflejen tres ejemplos usados para verificar el buen funcionamiento del algoritmo.
- Añadir a la documentación la representación gráfica de las variables calculadas y una discusión sobre su comportamiento.

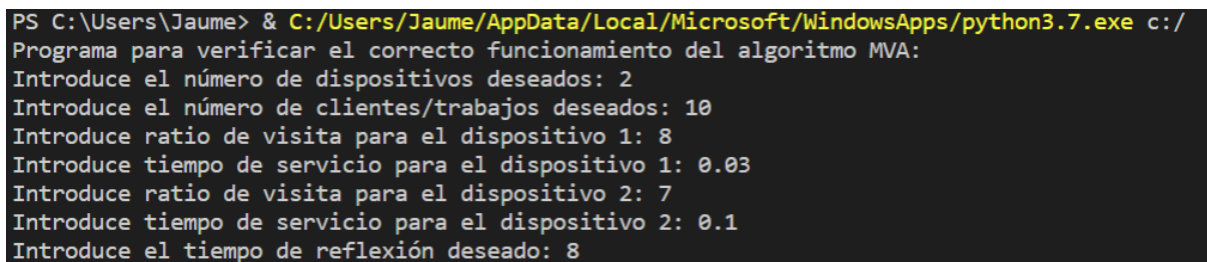
# 1. Introducción

El objetivo de esta práctica consiste en implementar el algoritmo MVA de redes cerradas y verificar su correcto funcionamiento. En mi caso, he elegido el lenguaje de programación Python por su facilidad de sintaxis y la cantidad de librerías como numpy, matplotlib u otras que permiten realizar los gráficos directamente sin tener que exportar los datos. En concreto, utilizo matplotlib.pyplot para los gráficos.

Vamos a ilustrar 3 casos de prueba, en los dos primeros comprobamos que los datos generados por el algoritmo son los correctos, mientras que en el último realizaremos un análisis a largo plazo ya que el número de trabajos será mayor. Además, dejaremos el código completo en el apartado contenido adicional por si es necesario ejecutar el programa.

## 2. Implementación del algoritmo

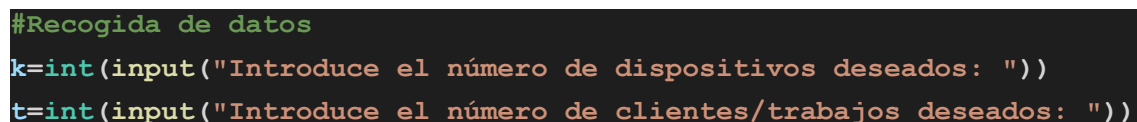
El algoritmo que hemos implementado se compone de las diferentes funciones que aparecen a continuación, con parámetros de  $i, N$ , siendo  $i$  el número de dispositivo y  $N$  el número de trabajos. En primer lugar, pedimos los datos al usuario mediante la instrucción `input`, que se encarga de la lectura por teclado:



```
PS C:\Users\Jaume> & C:/Users/Jaume/AppData/Local/Microsoft/WindowsApps/python3.7.exe c:/Programa para verificar el correcto funcionamiento del algoritmo MVA:
Introduce el número de dispositivos deseados: 2
Introduce el número de clientes/trabajos deseados: 10
Introduce ratio de visita para el dispositivo 1: 8
Introduce tiempo de servicio para el dispositivo 1: 0.03
Introduce ratio de visita para el dispositivo 2: 7
Introduce tiempo de servicio para el dispositivo 2: 0.1
Introduce el tiempo de reflexión deseado: 8
```

Figura 2.1: introducción de datos del ejemplo 2.

Los datos se almacenan en variables que serán usadas posteriormente para el cálculo de las funciones especificadas a continuación. Veamos el código a continuación:



```
#Recogida de datos
k=int(input("Introduce el número de dispositivos deseados: "))
t=int(input("Introduce el número de clientes/trabajos deseados: "))
```

```

for i in range(k):
    vRatio.append(int((input("Introduce ratio de visita para el
dispositivo "+str((i+1))+": "))))
    sTime.append(float((input("Introduce tiempo de servicio para el
dispositivo "+str(i+1)+" : "))))

z=int(input("Introduce el tiempo de reflexión deseado: "))

```

En cuanto a las variables, **k** representa el número de dispositivos, **t** el número de trabajos, **vRatio** y **sTime** son dos vectores para almacenar las razones de visita y los tiempos de servicio y **z** es el tiempo de reflexión deseado. Hemos almacenado todos los datos necesarios en variables que serán utilizadas para posteriores cálculos.

## 2.1 Función tiempo de respuesta total

La primera función se encarga de representar la expresión siguiente:

$$R(N) = \sum_{i=1}^k V_i * R_i(N)$$

Siendo k el número de dispositivos , realizamos un sumatorio de todas las razones de visita multiplicadas por el tiempo de respuesta del mismo dispositivo para el mismo trabajo.

Código:

```

def TotalResponseTime(N):
    i,x=0,0
    while i<k:
        x+=(vRatio[i]*ResponseTime_i(i,N))
        i+=1
    return x

```

## 2.2 Función tiempo de respuesta por estación

Al contrario de la anterior, esta calcula el tiempo de respuesta por dispositivo para N trabajos, reflejado en esta función:

$$R_i(N) = (N_i(N - 1) + 1) * S_i$$

Código:

```
def ResponseTime_i(i,N):  
    return (JobsNumber_i(i,N-1)+1)*sTime[i]
```

## 2.3 Función de productividad total

La productividad total se refleja de la siguiente manera:

$$X(N) = \frac{N}{Z+R(N)}$$

Donde N es el trabajo que recibe, Z el tiempo de reflexión y R(N) la función ya descrita anteriormente(2.1).

Código:

```
def Productivity(N):  
    return (N/(z+TotalResponseTime(N)))
```

## 2.4 Función de productividad por estaciones

La productividad por estación/dispositivo se compone así:

$$X_i(N) = X(N) * V_i$$

Siendo i el número de dispositivo, X(N) la productividad total(2.3) y V\_i la razón de visita correspondiente al dispositivo.

Código:

```
def Productivity_i(i,N):  
    return Productivity(N)*vRatio[i]
```

## 2.5 Función de utilización por estación

La expresión de utilización viene dada por la multiplicación de la productividad total del número de trabajos por la razón de visita y el tiempo de servicio del dispositivo i. Véase:

$$U_i(N) = X(N) * V_i * S_i$$

Código:

```
def Utilization_i(i,N):  
    return Productivity(N)*vRatio[i]*sTime[i]
```

## 2.7 Función número de trabajos por estación

Esta función nos calcula el número de trabajos por estación en función de los totales en el sistema. Si hay 0 trabajos totales, el resultado es 0:

$$N_i(0) = 0$$

Si no es el caso, sigue esta expresión:

$$N_i(N) = X(N) * V_i * R_i(N)$$

Siendo  $X(N)$  la productividad total,  $V_i$  la razón de visita del dispositivo  $i$  y  $R_i(N)$  el tiempo de respuesta por dispositivo en función de los trabajos.

Código:

```
def JobsNumber_i(i,N):  
    if (N==0):  
        return 0  
    return Productivity(N)*vRatio[i]*ResponseTime_i(i,N)
```

## 2.8 Visualización de los cálculos

Los resultados se muestran por consola mediante prints, de trabajo 1 hasta t, siguiendo la estructura a continuación:

TRABAJO 2:

|            |  |            |
|------------|--|------------|
| R1(0.0308) |  | N1(0.0548) |
| U1(0.0533) |  | X1(1.7775) |
| R2(0.1078) |  | N2(0.1677) |
| U2(0.1555) |  | X2(1.5553) |
| R(1.0013)  |  | X0(0.2222) |

### TRABAJO 3:

|            |  |            |
|------------|--|------------|
| R1(0.0311) |  | N1(0.0525) |
| U1(0.0506) |  | X1(1.6858) |
| R2(0.8553) |  | N2(1.2616) |
| U2(0.7376) |  | X2(1.4751) |
| R(6.2362)  |  | X0(0.2107) |

Estos mensajes se mostrarán tantas veces como trabajos se hayan introducido, dependiendo del número de dispositivos. El código siguiente hace esto posible:

```
#Visualización de los resultados
for j in range(t):#para los trabajos
    print("TRABAJO "+str(j+1)+":\n")
    for i in range(k):#para los dispositivos
        #visualizar R_i y N_i
        print("R"+str(i+1)+"("+str(round(ResponseTime_i(i,j+1),4))+") |
N"+str(i+1)+"("+
            str(round(JobsNumber_i(i,j+1),4))+")")
        #visualizar U_i y X_i
        print("U"+str(i+1)+"("+str(round(Utilization_i(i,j+1),4))+") |
X"+str(i+1)+"("+
            str(round(Productivity_i(i,j+1),4))+")")
        #visualizar R y X
        print(("R("+str(round(TotalResponseTime(j+1),4))+") "+"
X0("+str(round(Productivity(j+1),4))+")\n"))
```

## 2.9 Representación de funciones con matplotlib(pyplot)

Para poder observar el comportamiento de las funciones a largo plazo, las representaremos para un número determinado de trabajos, que será el eje X de nuestro gráfico. Veamos el proceso:

Primero importamos la librería correspondiente:

```
import matplotlib.pyplot as plt
```

A continuación definimos los valores en dos vectores, uno para el eje X y otro para el eje Y. Nosotros las definimos como AxisX y AxisY:

```
AxisX=[0,1,2,3,4,5,6,7,8,9,10] #eje X  
AxisY=[0,2,4,6,10,12,14,18,25] #eje Y
```

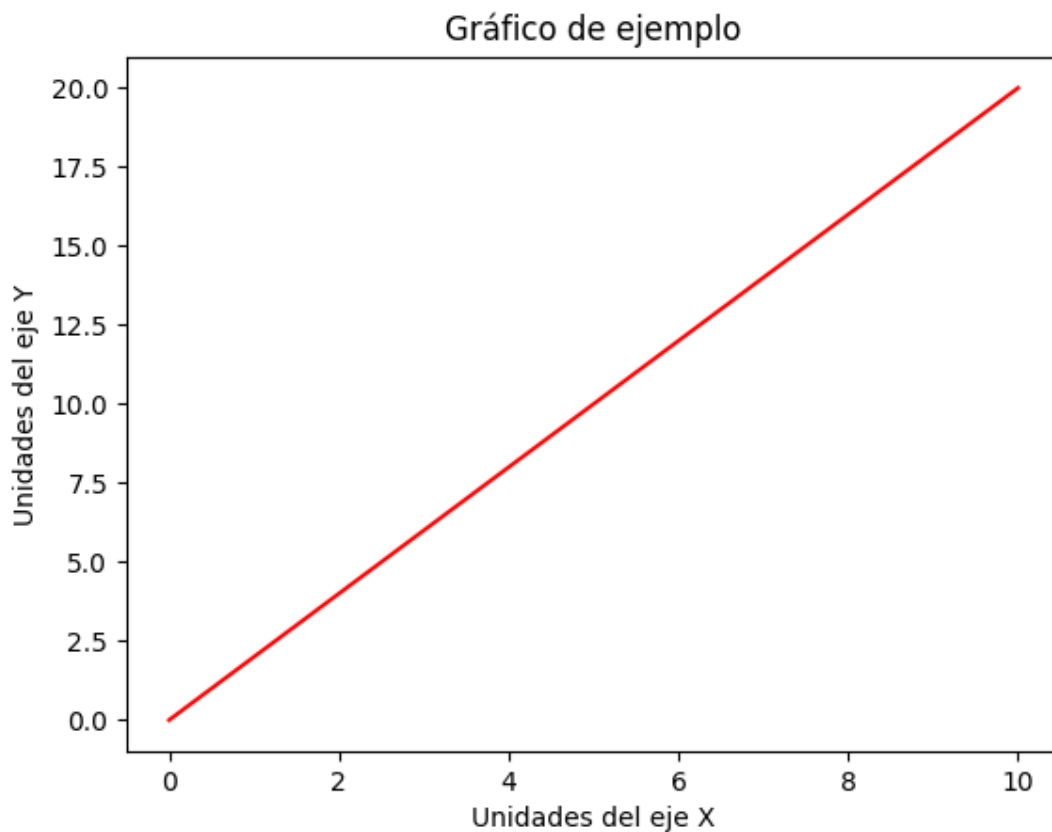
A continuación, mostramos mediante la instrucción plot el gráfico, siguiendo la estructura/sintaxis siguiente:

***plt.plot(eje x,eje y, color de la función)***

En nuestro caso:

```
plt.plot(AxisX,AxisY,color='r')  
plt.title("Gráfico de ejemplo")  
plt.xlabel("Unidades del eje X")  
plt.ylabel("Unidades del eje Y")  
plt.show()
```

Además, añadimos el título al gráfico mediante plt.title y unas etiquetas a los ejes mediante xlabel y ylabel. Finalmente, mostramos el gráfico que se despliega en una ventana emergente:



### 3.Casos de prueba

Para comprobar el correcto funcionamiento de nuestro algoritmo cogeremos tres casos de prueba y comprobaremos que la tabla de datos obtenida es similar a la ya vista. Además, graficaremos las funciones para ver su comportamiento a largo plazo en función del número de trabajos.

#### 3.1 Ejemplo 1(página 136,137)

En el primer ejemplo extraído del libro se nos especifican dos dispositivos, uno con razón de visita de 15 y tiempo de servicio 0.03 segundos, mientras que el otro tiene una razón de 14 y un tiempo de servicio de 0.5 segundos. Estos son los datos iniciales:

$$k = 2$$

$$t = 3$$

$$V = [15, 14]$$

$$S = [0.03, 0.5]$$

$$z = 5$$

Siendo k el número de dispositivos, t el número de trabajos y  $V_i, S_i$  la razón de visita y tiempo de servicio correspondientes al dispositivo i. Por ejemplo, el primer índice corresponde a la razón de visita del primer dispositivo(15). Veamos los resultados obtenidos

|           | R<br>(s) | R1<br>(s) | R2<br>(s) | N1<br>(t) | N2<br>(t) | X1<br>(t/s) | X2<br>(t/s) | U1<br>(0-1) | U2<br>(0-1) | X<br>(t/s) |
|-----------|----------|-----------|-----------|-----------|-----------|-------------|-------------|-------------|-------------|------------|
| Trabajo 1 | 7.45     | 0.03      | 0.5       | 0.0361    | 0.5622    | 1.2048      | 1.1245      | 0.0361      | 0.5622      | 0.0803     |
| Trabajo 2 | 11.402   | 0.0311    | 0.7811    | 0.0569    | 1.3335    | 1.829       | 1.7071      | 0.0549      | 0.8536      | 0.1219     |
| Trabajo 3 | 16.809   | 0.0317    | 1.1667    | 0.0654    | 2.2468    | 2.0633      | 1.9257      | 0.0619      | 0.9629      | 0.1376     |

Obtenidos todos los datos, vamos a graficarlos para ver cómo es su comportamiento:

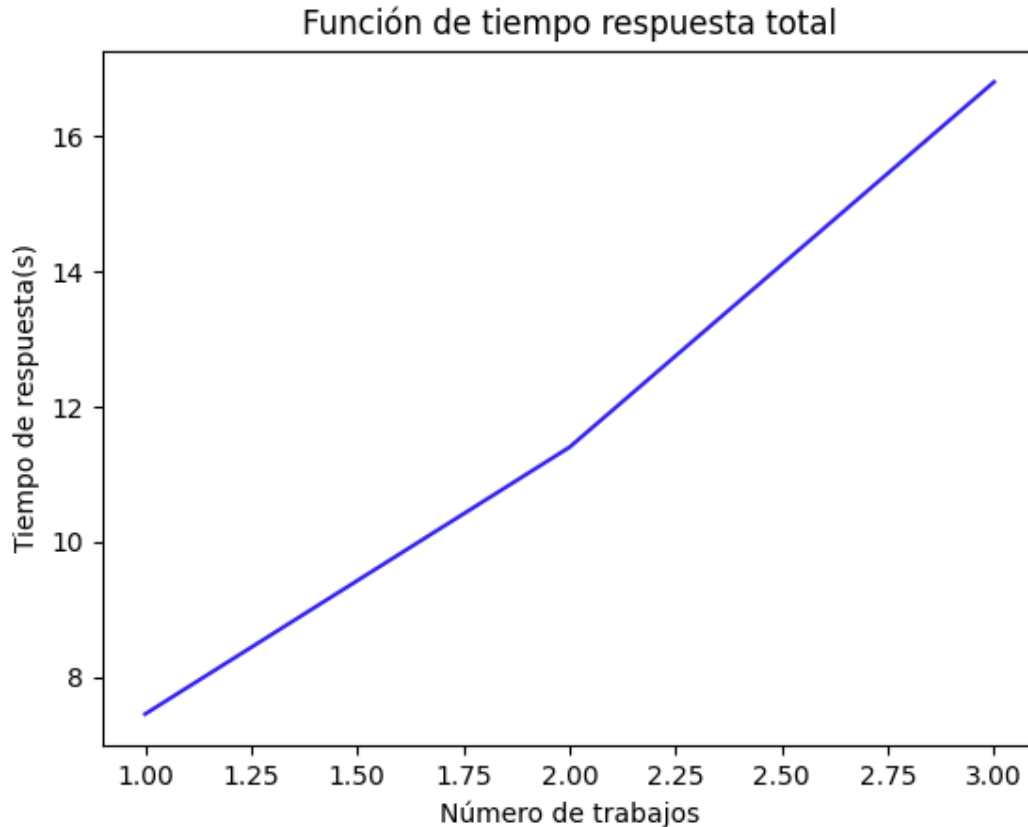
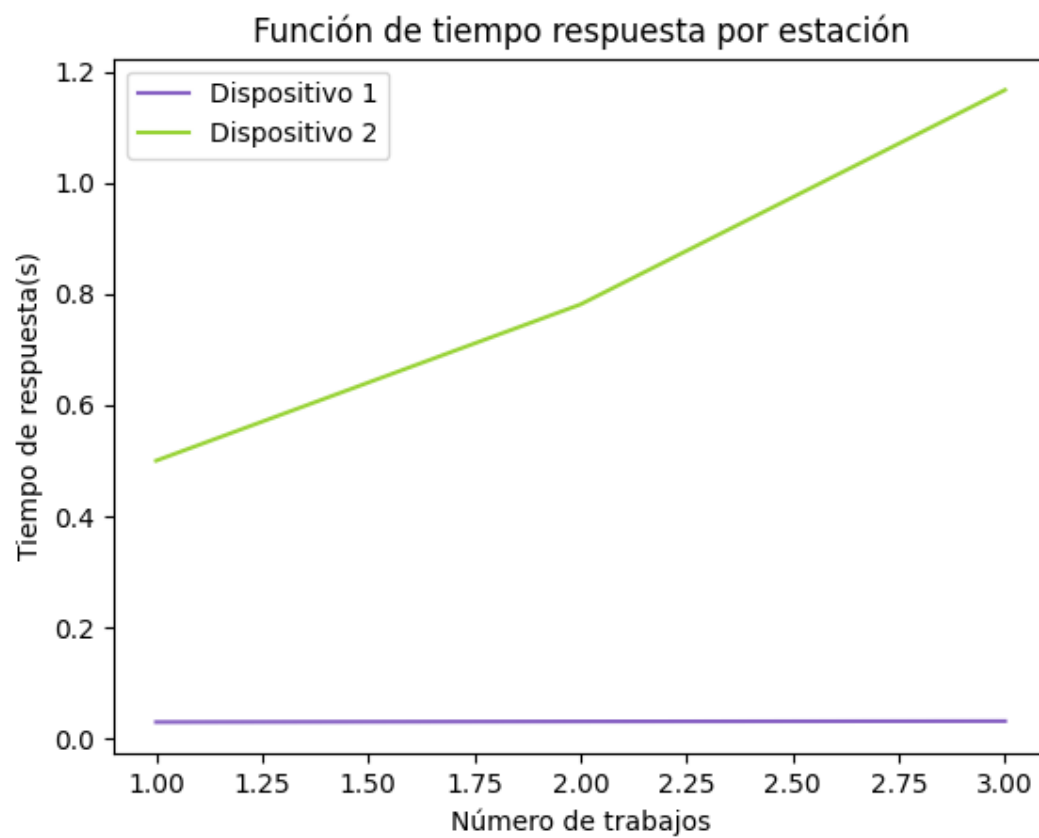


Figura 1: evolución tiempo de respuesta total en función de los trabajos



*Figura 2: evolución tiempo de respuesta por dispositivo*

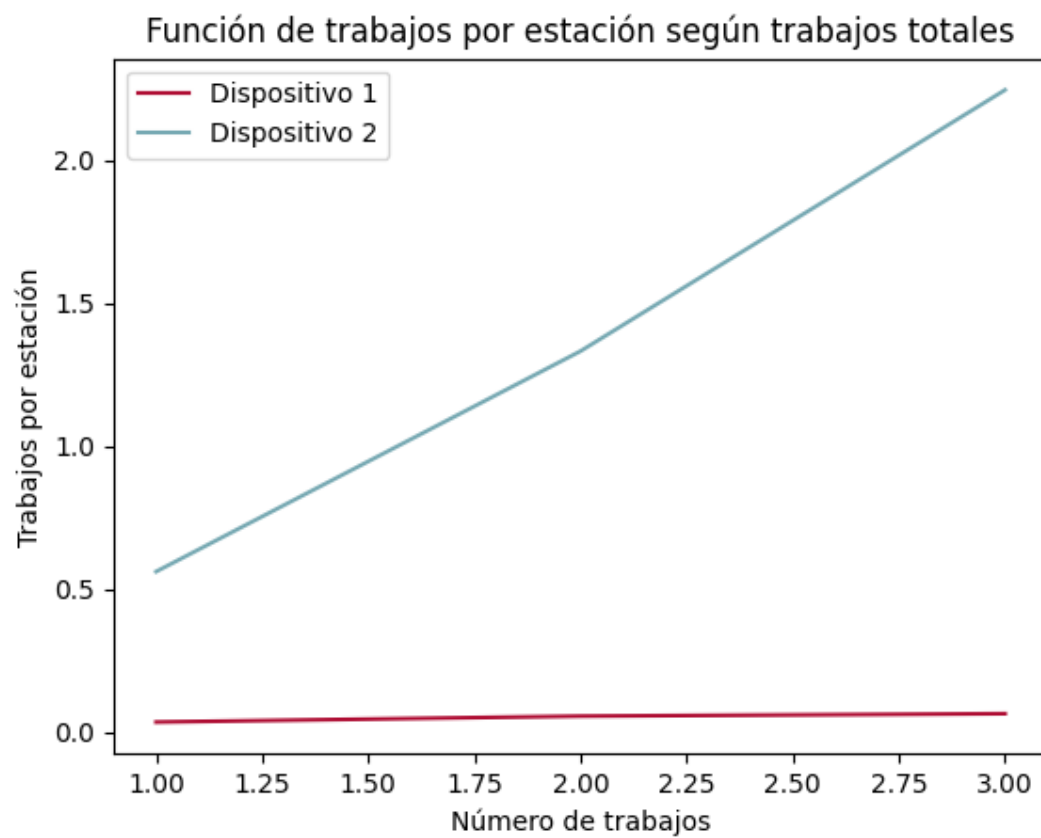
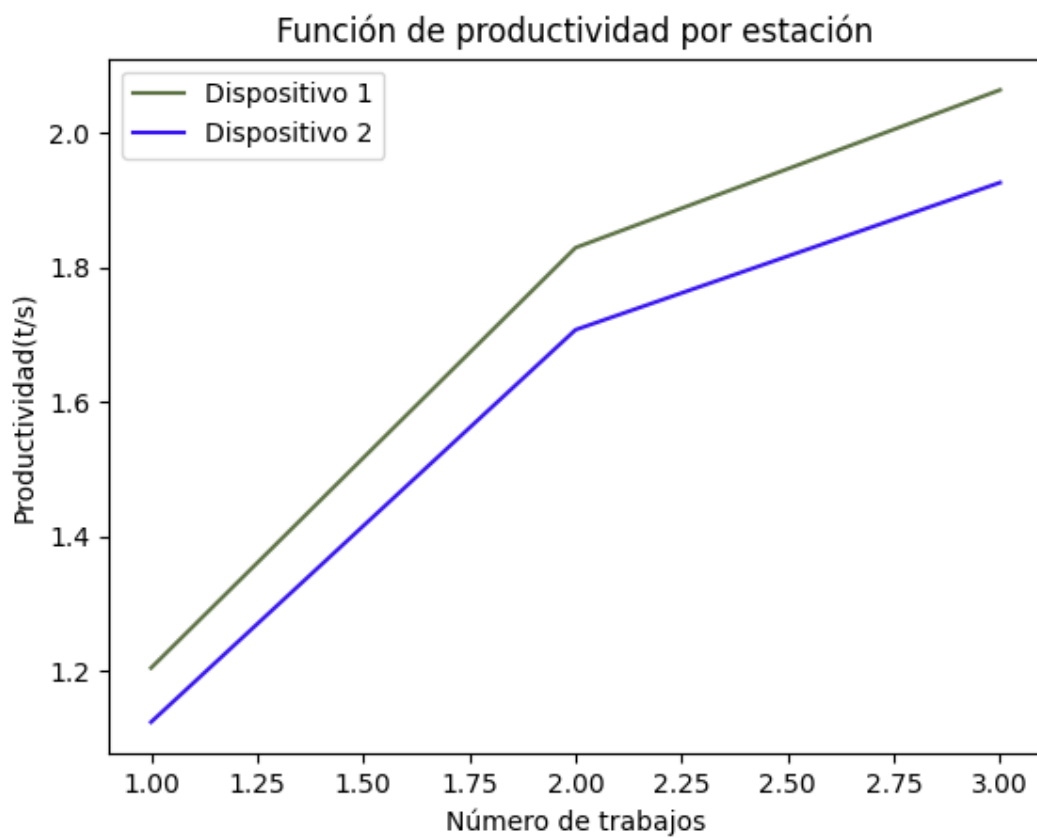


Figura 3: evolución trabajos por estación según trabajos totales en el sistema.



*Figura 4: evolución productividad por estación*

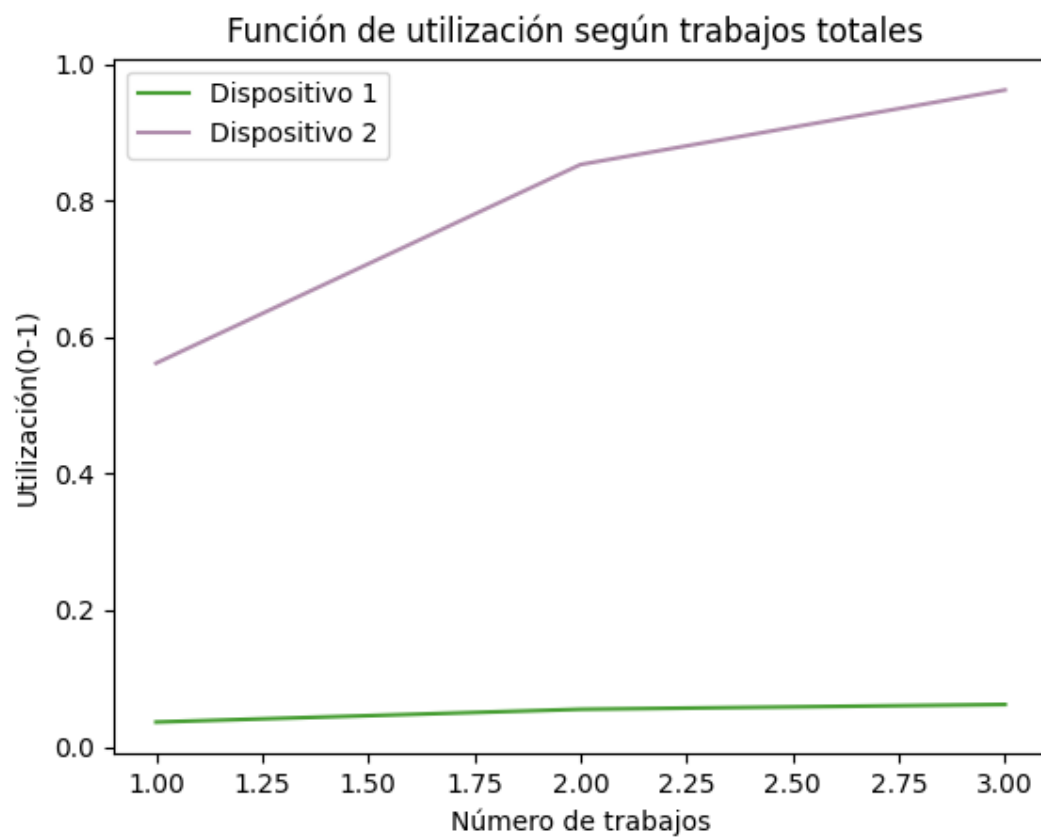
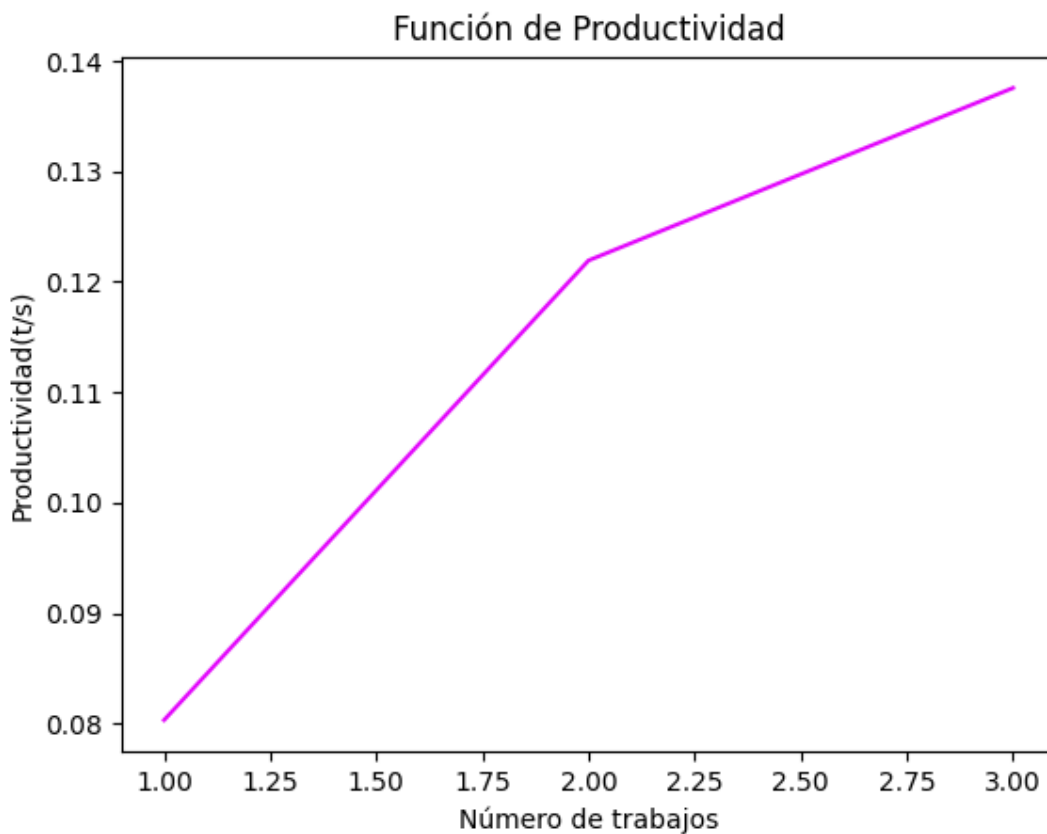


Figura 5: evolución utilización por dispositivos según trabajos



*Figura 6: evolución productividad total*

### 3.2 Ejemplo 2(ejercicio 5.1 página 144,145)

Si vamos a la página 144 del libro, concretamente en el ejercicio 5.1, encontramos otro caso a comprobar. Tenemos dos dispositivos, concretamente un procesador y un disco. El primero tiene una razón de visita de 8 y un tiempo de servicio de 0.03, mientras que el segundo es de 7 y 0.1 respectivamente. El sistema se compone de 10 usuarios, con un tiempo de reflexión de 8 segundos.

$$k = 2$$

$$t = 10$$

$$V = [8, 7]$$

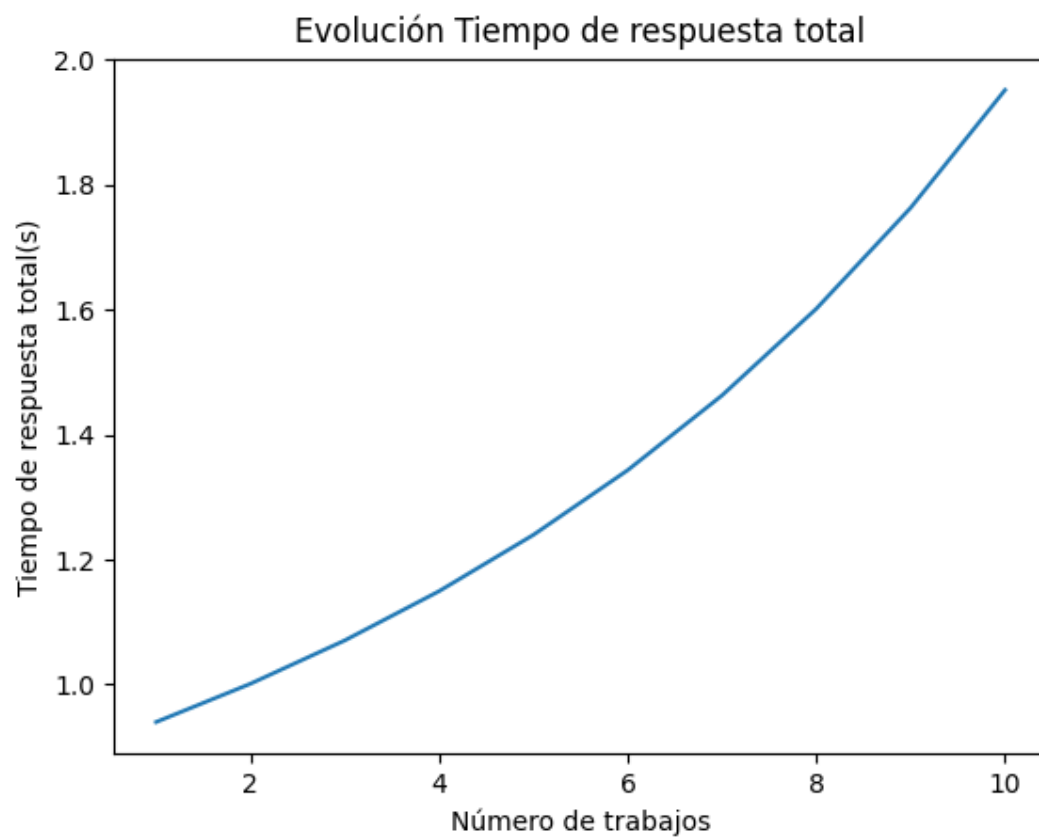
$$S = [0.03, 0.1]$$

$$z = 8$$

Se nos pide comprobar el correcto funcionamiento mediante nuestro algoritmo. Veamos los resultados:

|            | R<br>(s) | R1<br>(s) | R2<br>(s) | N1<br>(t) | N2<br>(t) | X1<br>(t/s) | X2<br>(t/s) | U1<br>(0-1) | U2<br>(0-1) | X<br>(t/s) |
|------------|----------|-----------|-----------|-----------|-----------|-------------|-------------|-------------|-------------|------------|
| Trabajo 1  | 0.94     | 0.03      | 0.1       | 0.0268    | 0.0783    | 0.8949      | 0.783       | 0.026       | 0.078       | 0.1119     |
| Trabajo 2  | 1.0013   | 0.0308    | 0.1078    | 0.0548    | 0.1677    | 1.7775      | 1.5553      | 0.0533      | 0.1555      | 0.2222     |
| Trabajo 3  | 1.0705   | 0.0316    | 0.1168    | 0.0837    | 0.2703    | 2.6459      | 2.3152      | 0.0794      | 0.2315      | 0.3307     |
| Trabajo 4  | 1.1493   | 0.0325    | 0.127     | 0.1137    | 0.3888    | 3.4975      | 3.0603      | 0.1049      | 0.306       | 0.4372     |
| Trabajo 5  | 1.2394   | 0.0334    | 0.1389    | 0.1446    | 0.5261    | 4.3293      | 3.7881      | 0.1229      | 0.3788      | 0.5412     |
| Trabajo 6  | 1.343    | 0.0343    | 0.1526    | 0.1764    | 0.686     | 5.1376      | 4.4954      | 0.1541      | 0.4495      | 0.6422     |
| Trabajo 7  | 1.4626   | 0.0353    | 0.1686    | 0.2089    | 0.8731    | 5.9181      | 5.1783      | 0.1775      | 0.5178      | 0.7398     |
| Trabajo 8  | 1.6013   | 0.0363    | 0.1873    | 0.2417    | 1.0925    | 6.6658      | 5.8326      | 0.2         | 0.5833      | 0.8332     |
| Trabajo 9  | 1.7628   | 0.0373    | 0.2092    | 0.2747    | 1.3503    | 7.375       | 6.4531      | 0.2212      | 0.6453      | 0.9219     |
| Trabajo 10 | 1.9511   | 0.0382    | 0.235     | 0.3074    | 1.6533    | 8.0393      | 7.0344      | 0.2412      | 0.7034      | 1.0049     |

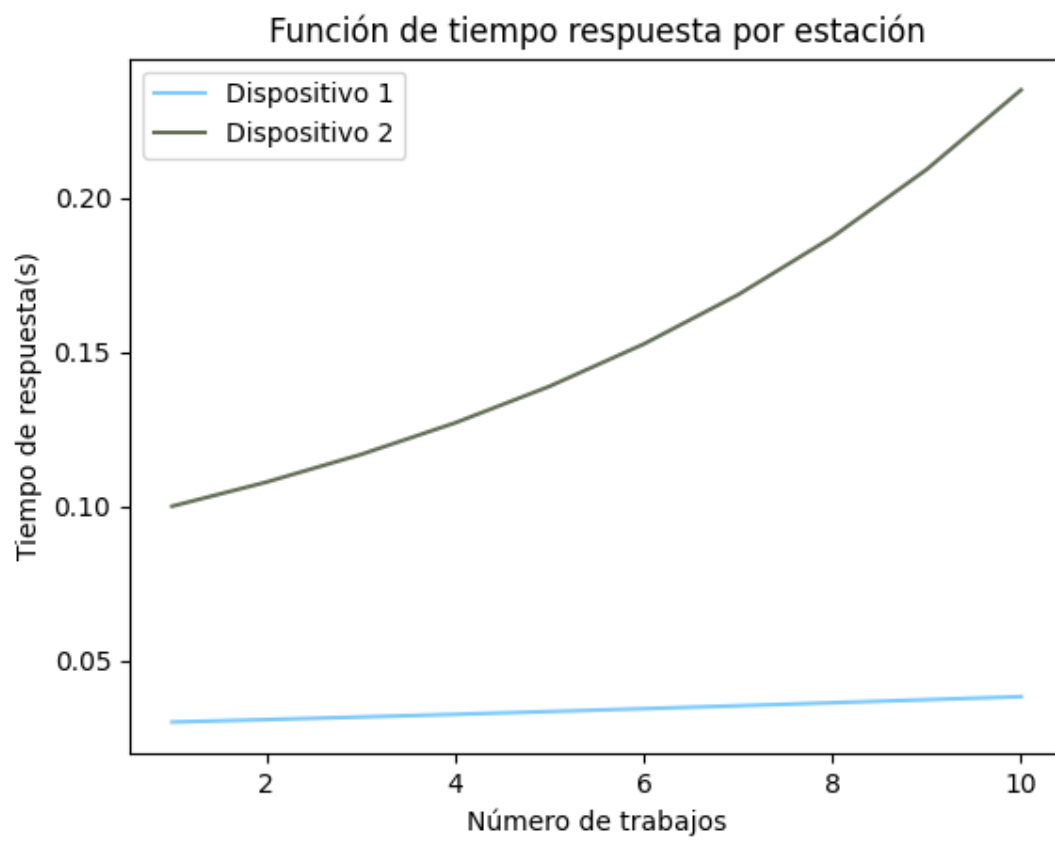
Ahora, veámoslo de una manera mucho más gráfica:



*Figura 1: evolución tiempo de respuesta total en función de los trabajos*



*Figura 2: evolución productividad total*



*Figura 3: tiempo de respuesta por estación en función de los trabajos*

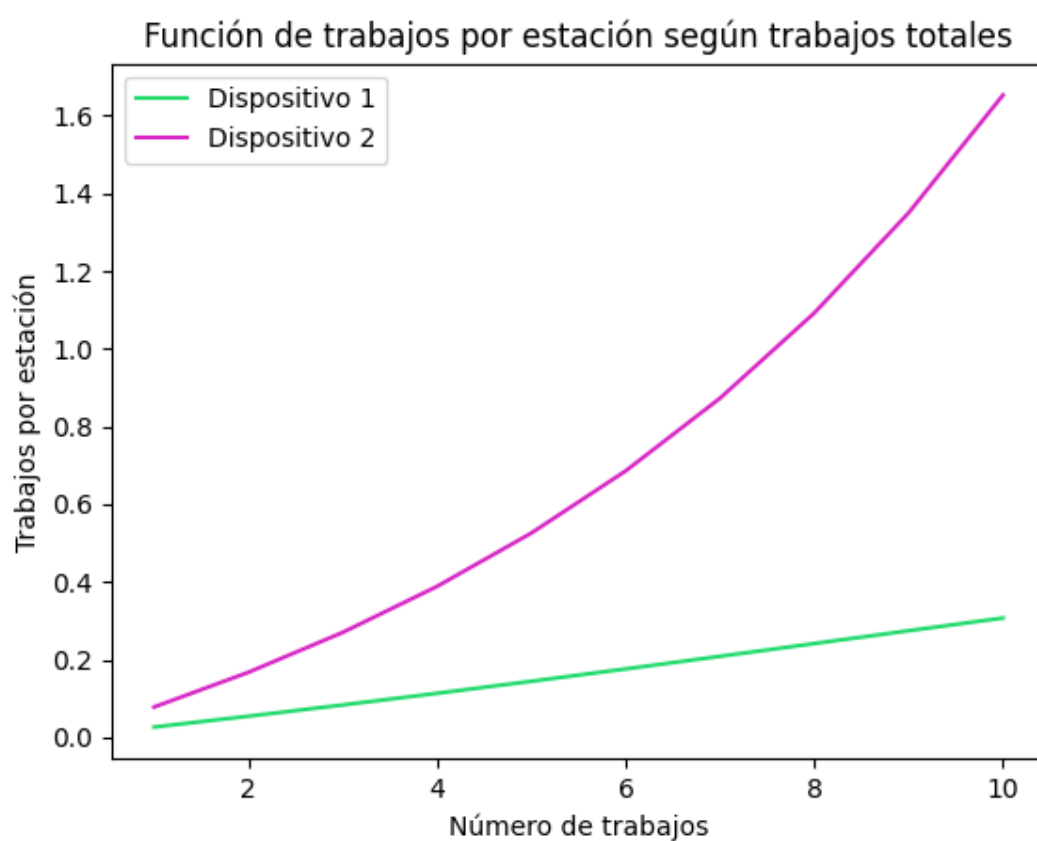


Figura 4: número de trabajos por estación en función de trabajos en el sistema.

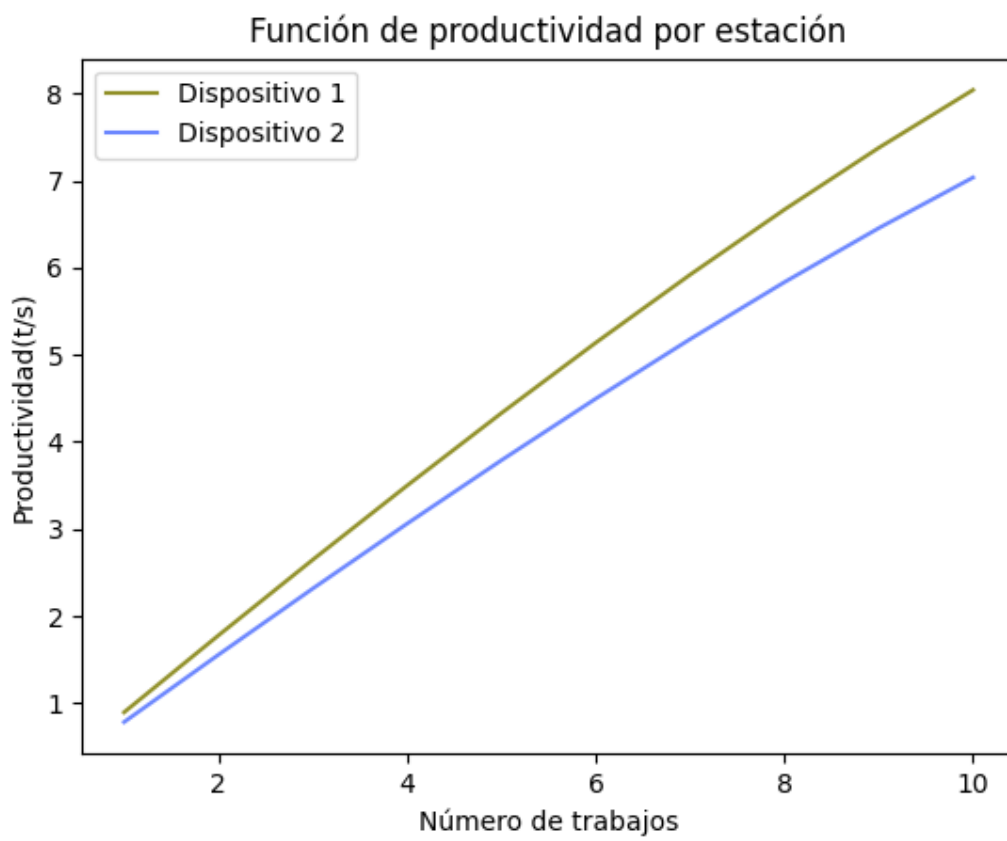
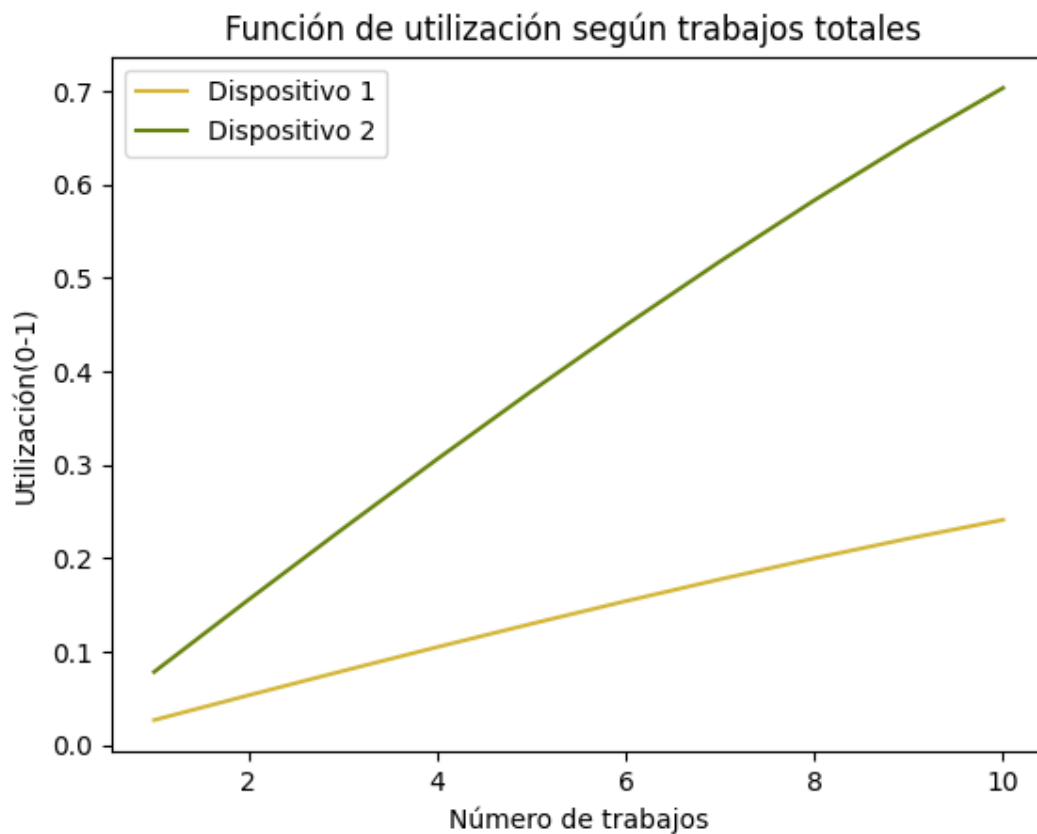


Figura 5: productividad por estación en función de los trabajos



*Figura 6: utilización por estación medido en intervalo de 0-1(porcentaje)*

### 3.3 Ejemplo 3 (personalizado)

Como último ejemplo, vamos a personalizar el primero ya visto de manera que se pueda observar el comportamiento a largo plazo. Por tanto, pondremos que el número de trabajos sea 15. Supongamos estos datos iniciales:

$$k = 2$$

$$t = 15$$

$$V = [15, 14]$$

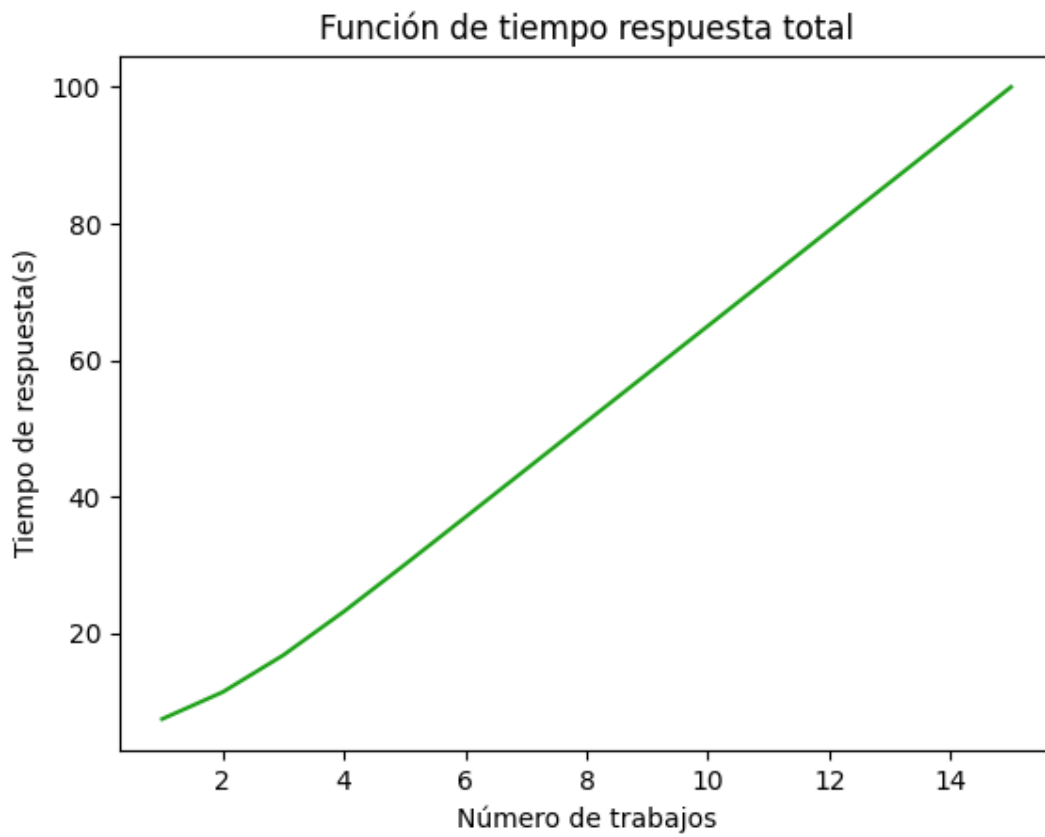
$$S = [0.03, 0.5]$$

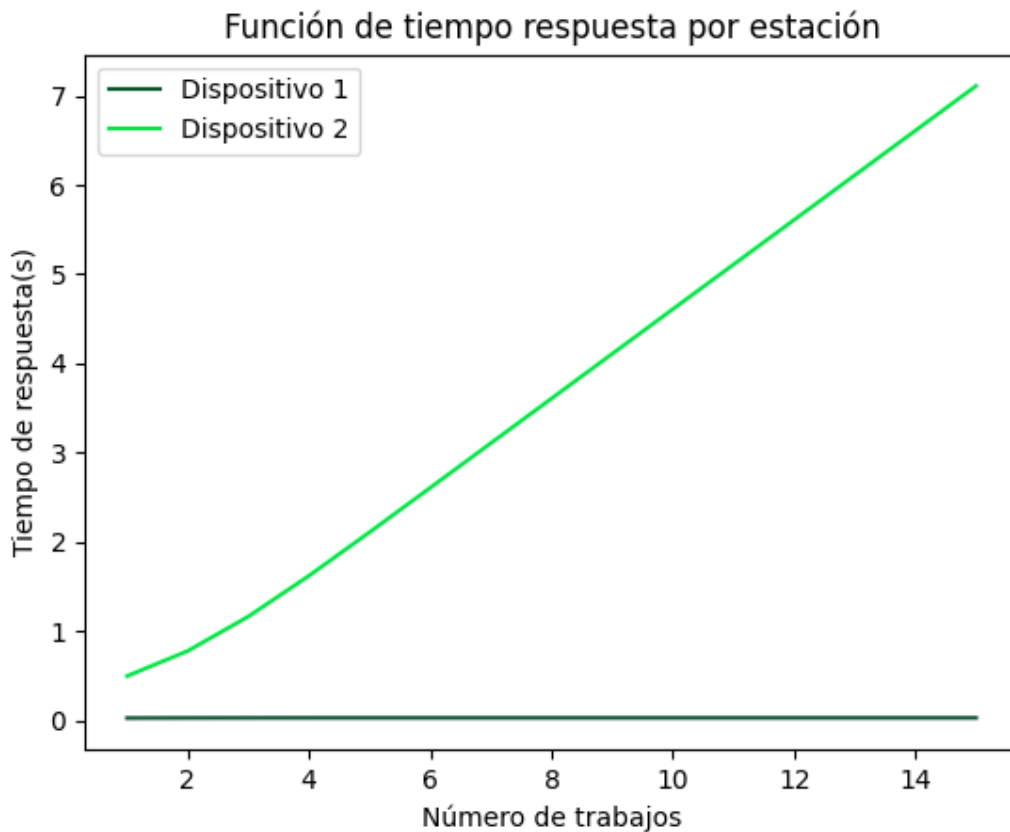
$$z = 5$$

Como ya hemos introducido los datos importantes en la tabla del primer enunciado, solo graficaremos las variables calculadas y analizaremos su comportamiento. Cabe destacar que este ejemplo es una ampliación del primero para 15 trabajos, con el fin de así poder observar más los límites y comportamiento de las variables a largo plazo.

Aquí añadiremos un análisis de cada gráfico, comentando que observaciones se han apreciado en el comportamiento de las variables.

### 3.3.1 Análisis tiempo de respuesta total y por dispositivo





Como podemos observar en ambos gráficos, vemos que el tiempo de respuesta tiende a infinito a medida que van aumentando los trabajos. Si observamos la expresión, vemos que el tiempo de respuesta depende del número de trabajos, multiplicado por su tiempo de servicio. El total se obtiene sumando todos los tiempos de respuesta por su razón de visita correspondiente al número de dispositivo.

$$R(N) = \sum_{i=1}^k V_i * R_i(N)$$

$$R_i(N) = (N_i(N - 1) + 1) * S_i$$

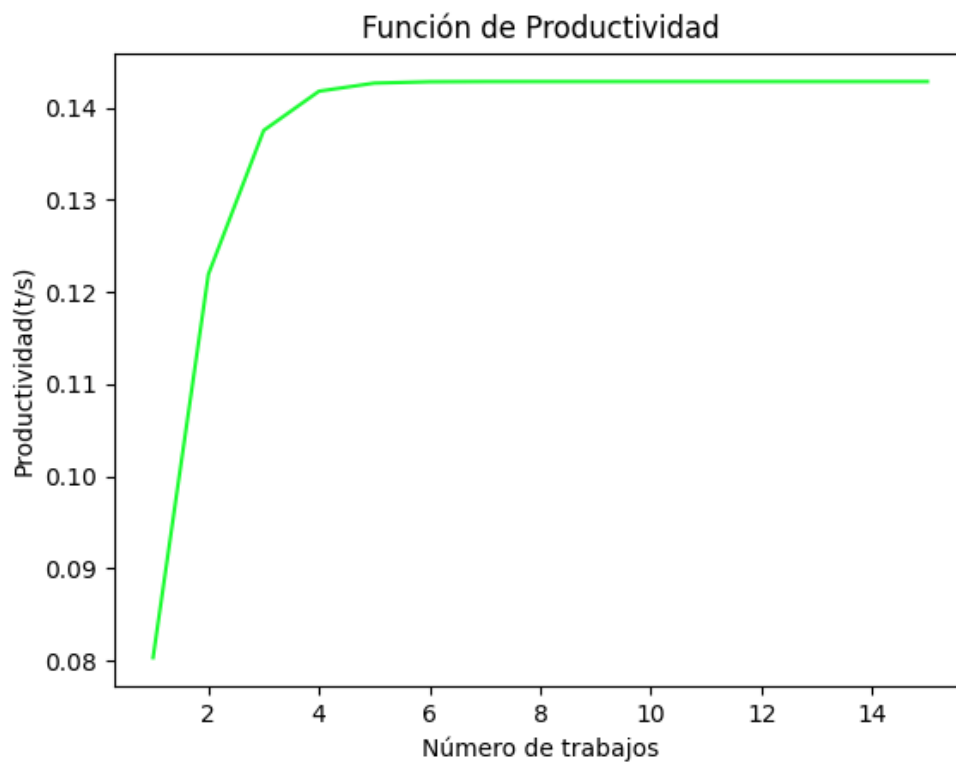
Esto nos permite sacar conclusiones y apreciar que el primer dispositivo con un tiempo de servicio de 0.03 segundos casi no crece, debido a la gran diferencia entre el tiempo de servicio del primer dispositivo 0.5, que es unas 17 veces más grande que el primero. Por la razón de que es tantas veces más grande que el primero, experimenta un mayor crecimiento y tiende a infinito, observándose de una

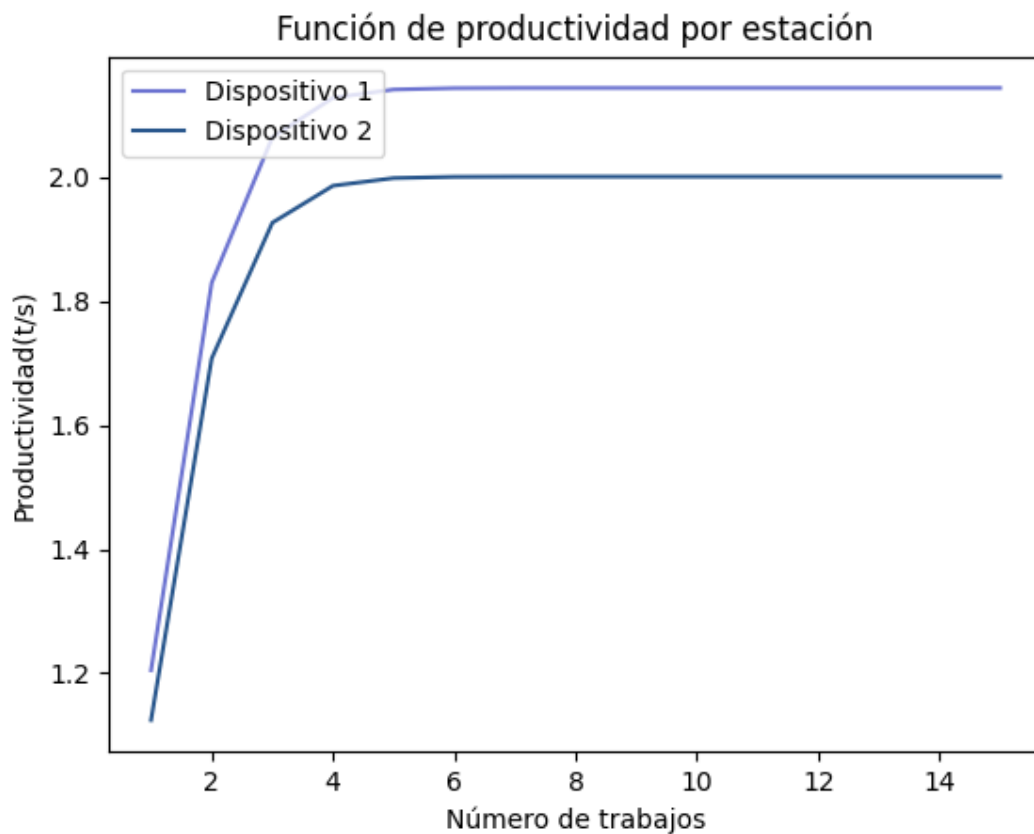
manera mucho más visual.

Véase la relación entre ambos tiempos de servicio:

$$S_1 = 16.66 \cdot S_2$$

### 3.3.2 Análisis productividad total y por dispositivos





Para poder analizar el comportamiento y explicar sus causas, veamos las expresiones correspondientes a las gráficas:

$$X_i(N) = X(N) * V_i$$

$$X(N) = \frac{N}{Z + R(N)}$$

Como podemos observar, vemos que la productividad total depende del tiempo de reflexión, trabajos y el tiempo de respuesta. Si el número de trabajos es mayor, la productividad será mayor. Sin embargo, si el tiempo para procesar un trabajo( la suma del tiempo de respuesta más el tiempo de reflexión) aumenta, se realizarán menos trabajos por unidad de tiempo. Por tanto, tenemos dos dependencias, el número de trabajos y el tiempo necesario

por trabajo.

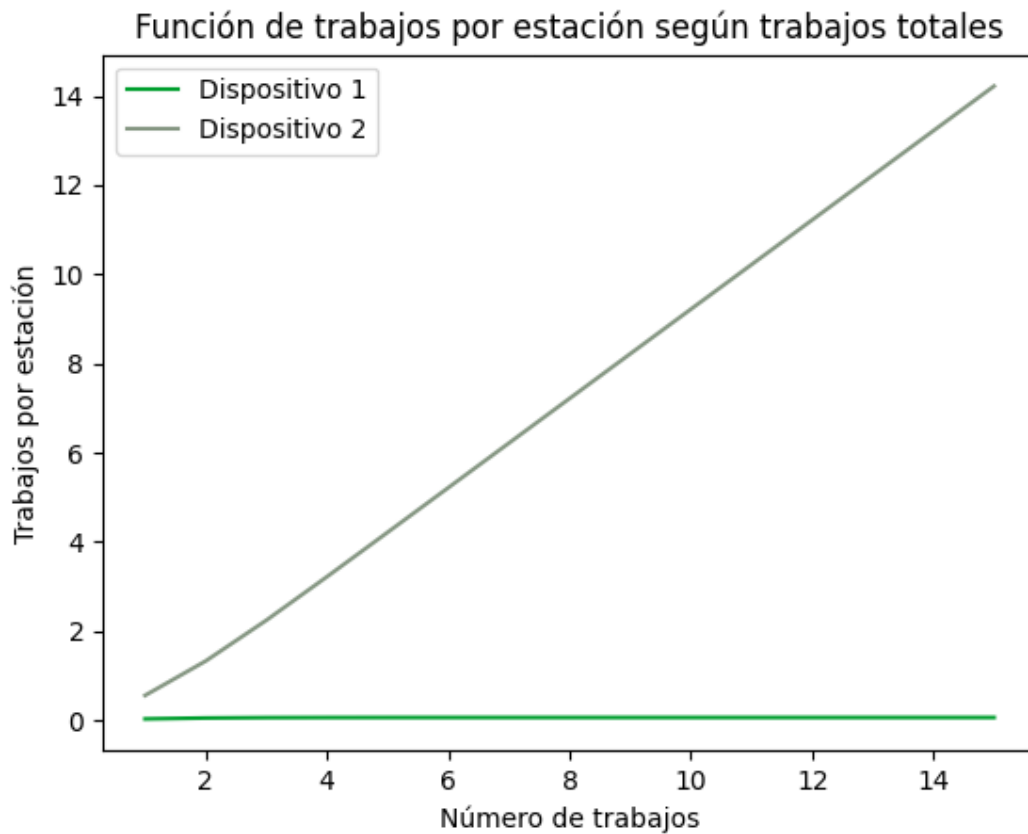
Si observamos la gráfica vemos que la productividad llega a un máximo, por lo tanto, concluimos que si se estanca es por que las variables  $R(N)$  y  $N$  van ligadas (el tiempo de reflexión no cambia). Es decir, la productividad aumenta si el número de trabajos es mayor, aunque a mayor número de trabajos también aumenta el tiempo de respuesta. El aumento de los trabajos se equilibra con el aumento del tiempo de respuesta. Además, podemos decir que la productividad por dispositivo es la multiplicación de este valor por la razón de visita, motivo principal por el cual el dispositivo 1 es ligeramente mayor, ya que tiene una razón de visita de 15 trabajos, mientras que el segundo es de 14.

En conclusión, tenemos estos factores que explican el comportamiento de esta función:

**-1.** A mayor número de trabajos, mayor productividad. Sin embargo, se equilibra con el tiempo de respuesta, que a la vez también aumenta. El tiempo de reflexión no cambia.

**-2.** La productividad por dispositivo depende de su razón de visita, ya que  $X(N)$  es común en ambos. A mayor razón de visita, mayor productividad que otros dispositivos.

### 3.3.3 Análisis trabajos por estación



Siguiendo la metodología anterior, primero observaremos la expresión de la función para después determinar su comportamiento y encontrar las causas de este. Veamos la función correspondiente a este gráfico:

$$N_i(N) = X(N) * V_i * R_i(N)$$

Esta función es más compleja ya que depende de otras dos funciones más. En resumen, la productividad llega a un máximo y la razón de visita no cambia. Por tanto, si tenemos una multiplicación de 3 valores y dos de ellos no tienden a infinito, para que la función crezca de esa forma debemos multiplicar por un valor que no sea fijo y aumente hasta infinito. De esa manera, sacamos la conclusión de que esta función tiende a infinito debido al tiempo de respuesta. Veámoslo de una manera más sencilla:

$$F(N) = k_1 * k_2 * R_i(N)$$

De esta manera,  $k_1$  indica el valor máximo que coge la función de la productividad,  $k_2$  es fijo debido a que la razón de visita no cambia y  $R_i(N)$  va aumentando, dándole a la función general un crecimiento hasta infinito.

### 3.3.4 Análisis Utilización Dispositivos



Veamos la fórmula de la utilización y sus dependencias:

$$U_i(N) = X(N) * V_i * S_i$$

$$X(N) = \frac{N}{Z+R(N)}$$

Como podemos observar, la utilización sigue un comportamiento similar a la productividad, cosa que no nos sorprende ya que esta depende de la anterior. Si la productividad sigue una asíntota horizontal, la utilización tiende al mismo comportamiento, ya que es esta misma multiplicada por la razón de visita y su tiempo de servicio. No cabe discutir tan profundamente esta función ya que es muy similar a la productividad, que llega a un límite ya que  $N$  crece junto a  $R(N)$ . Debido a la diferencia entre los tiempos de servicio, vemos que el dispositivo 2 llega antes al 100% de utilización. Hemos dicho anteriormente que el tiempo de servicio del dispositivo 2 es 17 veces mayor que el primero, factor que lo convierte en el cuello de botella. Mientras que el dispositivo 1 apenas llega a un 10% de utilización, el segundo ya llega al 100%. Esto nos demuestra que el rendimiento del sistema viene limitado por el segundo componente. Por tanto podemos concluir que el segundo dispositivo actúa como factor limitante, mejor dicho cuello de botella.

## 4. Conclusión

La elaboración de esta práctica ha sido en Python ya que es un lenguaje más integrado en el ámbito matemático, con librerías que permiten hacer gráficos directamente sin exportar los datos. Sin embargo, es un lenguaje interpretado, cosa que hace reducir la velocidad de ejecución al no ser compilado como Java o C. Por tanto, tenemos una mayor facilidad a la hora de programar aunque el tiempo de ejecución será menor.

En cuanto a los datos, podemos decir claramente que analizar la función según su expresión nos ayuda a determinar su comportamiento y ver las causas de este. Además, obtenemos conclusiones muy interesantes como por ejemplo determinar que dispositivo actúa como factor limitante(cuello de botella). Además, la función nos ayuda a poder ver su crecimiento de una manera mucho más visual.

## 5. Contenido adicional

Aquí adjunto todo el código en caso de querer ser ejecutado:

```
import numpy as np
import matplotlib.pyplot as plt
```

```

import random as rand

#Arrays necesarios para el ratio de visitas y tiempo de servicio
vRatio =[]
sTime  =[]

def ResponseTime_i(i,N):
    return (JobsNumber_i(i,N-1)+1)*sTime[i]

def TotalResponseTime(N):
    i,x=0,0
    while i<k:
        x+=(vRatio[i]*ResponseTime_i(i,N))
        i+=1
    return x

def JobsNumber_i(i,N):
    if (N==0):
        return 0
    return Productivity(N)*vRatio[i]*ResponseTime_i(i,N)

def Productivity(N):
    return (N/(z+TotalResponseTime(N)))

def Productivity_i(i,N):
    return Productivity(N)*vRatio[i]

def Utilization_i(i,N):
    return Productivity(N)*vRatio[i]*sTime[i]

#MAIN
print("Programa para verificar el correcto funcionamiento del
algoritmo MVA:")

#Recogida de datos
k=int(input("Introduce el número de dispositivos deseados: "))
t=int(input("Introduce el número de clientes/trabajos deseados:
"))
for i in range(k):

```

```

        vRatio.append(int((input("Introduce ratio de visita para el
dispositivo "+str((i+1))+": "))))
        sTime.append(float((input("Introduce tiempo de servicio para
el dispositivo "+str(i+1)+": "))))

    z=int(input("Introduce el tiempo de reflexión deseado: "))

    #Visualización de los resultados
    for j in range(t):#para los trabajos
        print("TRABAJO "+str(j+1)+":\n")
        for i in range(k):#para los dispositivos
            #visualizar R_i y N_i

print("R"+str(i+1)+"("+str(round(ResponseTime_i(i,j+1),4))+")"
N"+str(i+1)+"("
            str(round(JobsNumber_i(i,j+1),4))+")")
            #visualizar U_i y X_i

print("U"+str(i+1)+"("+str(round(Utilization_i(i,j+1),4))+")"
X"+str(i+1)+"("
            str(round(Productivity_i(i,j+1),4))+")")
            #visualizar R y X
            print(("R"+str(round(TotalResponseTime(j+1),4))+")"+"
X0("+str(round(Productivity(j+1),4))+")\n"))

x=[1,2,3,4,5,6,7,8,9,10]
func=[TotalResponseTime,ResponseTime_i,JobsNumber_i,Productivity_
i,Utilization_i,Productivity]#array de funciones
funcStr=["Tiempo de respuesta total(s)","Tiempo de respuesta del
dispositivo ","Número de trabajos en la estación ",
        "Productividad del dispositivo(t/s) ","Utilización del
dispositivo(0-1)","Productividad total"]#array titulo de funciones para
eje y
titleStr=["Evolución Tiempo de respuesta total","Evolución tiempo
de respuesta por estación","Número de trabajos por dispositivo",
        "Productividad por estación","Utilización por
estación","Productividad total"]#array titulos del gráfico

```

```

array=[]

#1Total responseTime
for i in range(1,len(x)+1):
    array.append(TotalResponseTime(i))
    print(str(i))
plt.plot(x,array,color=(rand.random(),rand.random(),rand.random())
))

plt.title("Función de tiempo respuesta total", fontdict=None,
loc='center', pad=None)
plt.xlabel("Número de trabajos")
plt.ylabel("Tiempo de respuesta(s)")
plt.show()

#2.response time
for j in range(k):
    del array[:]
    for i in range(1,len(x)+1):
        array.append(ResponseTime_i(j,i))

plt.plot(x,array,color=(rand.random(),rand.random(),rand.random()),label="Dispositivo "+str(j+1))

plt.title("Función de tiempo respuesta por estación",
fontdict=None, loc='center', pad=None)
plt.xlabel("Número de trabajos")
plt.ylabel("Tiempo de respuesta(s)")
plt.legend(loc="upper left")
plt.show()

# #3.mostrar TOTAL jobs
for j in range(k):
    del array[:]
    for i in range(1,len(x)+1):
        array.append(JobsNumber_i(j,i))

plt.plot(x,array,color=(rand.random(),rand.random(),rand.random()),label="Dispositivo "+str(j+1))

plt.title("Función de trabajos por estación según trabajos
totales", fontdict=None, loc='center', pad=None)
plt.xlabel("Número de trabajos")
plt.ylabel("Trabajos por estación")
plt.legend(loc="upper left")

```

```

plt.show()

# #4. mostrar productividad por estacion
for j in range(k):
    del array[:]
    for i in range(1,len(x)+1):
        array.append(Productivity_i(j,i))

plt.plot(x,array,color=(rand.random(),rand.random(),rand.random()),label="Dispositivo "+str(j+1))
    plt.title("Función de productividad por estación", fontdict=None,
loc='center', pad=None)
    plt.xlabel("Número de trabajos")
    plt.ylabel("Productividad(t/s)")
    plt.legend(loc="upper left")
    plt.show()

# #5. UTILIZACION X DISPOSITIVO
for j in range(k):
    del array[:]
    for i in range(1,len(x)+1):
        array.append(Utilization_i(j,i))

plt.plot(x,array,color=(rand.random(),rand.random(),rand.random()),label="Dispositivo "+str(j+1))
    plt.title("Función de utilización según trabajos totales ",
fontdict=None, loc='center', pad=None)
    plt.xlabel("Número de trabajos")
    plt.ylabel("Utilización(0-1)")
    plt.legend(loc="upper left")
    plt.show()

#6.Productividad
del array[:]
for i in range(1,len(x)+1):
    array.append(Productivity(i))
plt.plot(x,array,color=(rand.random(),rand.random(),rand.random()
))
    plt.title("Función de Productividad", fontdict=None,
loc='center', pad=None)
    plt.xlabel("Número de trabajos")

```

```
plt.ylabel("Productividad(t/s)")  
plt.show()
```